AutoBench

Architecture & Design Overview

A pure-Python, HTTP-only service that deploys and evaluates agent benchmarks across multiple cluster-specific Rossoctl instances.

Agenda

What this deck covers, in order

1. **Overview & key design decisions**
   what the Service is, and the seven choices that shape it
2. **Auto-benchmarking design motif**
   the shape of the system, before the wiring detail
3. **Architecture**
   client, Service, and the per-cluster instance selected by JWT iss
4. **Architecture with workload specific components**
   inside the workload: sidecar, user simulator, IBAC judge
5. **The two-token auth model**
   why the caller's token is never forwarded upstream
6. **Benchmark catalog & run lifecycle**
   the three benchmarks and the REST flow that drives them
7. **The three benchmarks — what they measure**
   7.1 the difficulty ladder · 7.2 what each stresses · 7.3 the traps
8. **The canonical 12-run matrix**
   8.1 what the 12 runs parameterize · 8.2 what they measured
9. **Cross-platform & plugin overhead**
   9.1 OpenShift vs KinD, like-for-like · 9.2 what AuthBridge costs
10. **What the Service can & cannot enact**
    the HTTP-only boundary, made explicit

1. Overview & Key Design Decisions

What it is

Deploys a benchmark's MCP tool + A2A agent, runs the evaluation, reads results, and exports them — all over HTTP.

Objectives

- Automate benchmarking lifecycle operations
- Cross-user/cluster sharing of benchmark run results
- Easy to configure in-cluster & cross-cluster benchmark runs
- Asynchronous parallel benchmark runs
- Secure, scalable, auditable & resilient

3 benchmarks

- gsm8k — single-turn: one prompt in, one answer out, no dialogue partner
- tau2 — multi-turn: a server-side user simulator LLM plays the customer
- appworld — long-horizon: many API calls across simulated apps

Client contract

- Point at a hostname, send Authorization: Bearer <caller JWT> — same from kind to prod

Key design decisions

Pure-Python, HTTPS-only to Rossoctl
No shell-out: smaller image, no injection surface, structured errors, testable.

Two-token auth model
Caller JWT attributes + routes only (never forwarded); Service mints its own benchmarker (ROPC) token to Rossoctl.

iss is the trust anchor
The JWT issuer selects the per-instance config — no separate instance argument, so no confused-deputy escape.

Per-request ROPC login
Fresh Service token every request — no expiry handling.

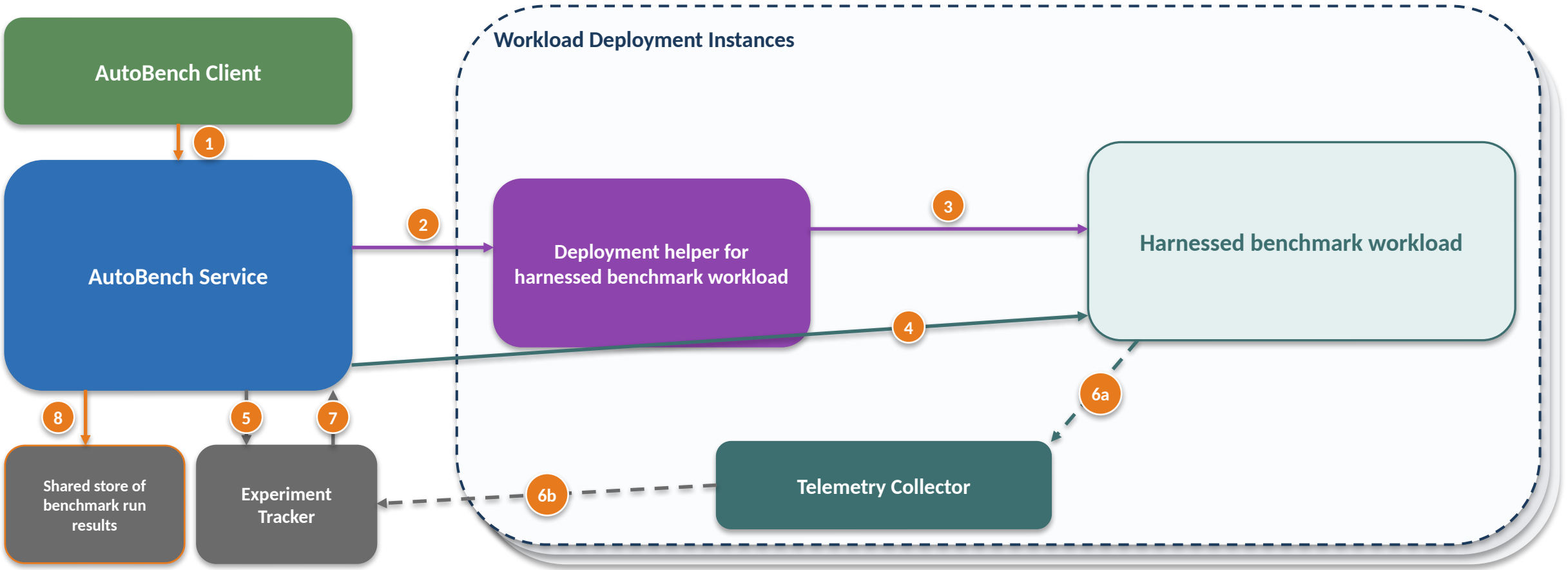
iss-keyed per-instance config
One file per issuer: Rossoctl URL, benchmarker cred, MLflow/S3, optional Keycloak backchannel + workload route templates.

Enact only what the API allows
Deploy/run/report/export — now including AuthBridge plugin presets (layer-3). Only workload Secrets remain out-of-band: prechecked and reported (424), never silently ignored.

MLflow with Service; S3 in the cloud
MLflow is co-located with the Service (per-service traces it emits + reads), not in the workload cluster; S3 is an external cloud service — the shared cross-service sink.

2. Auto-Benchmarking Design Motif

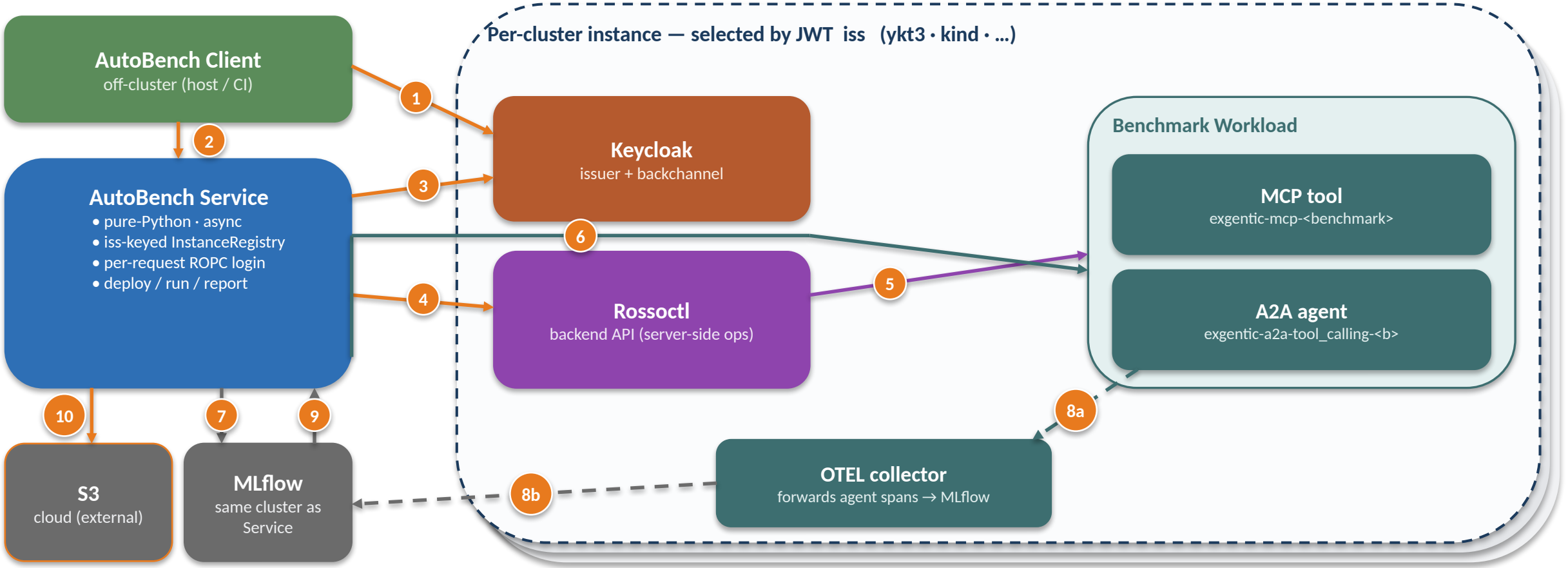
One client, one Service, N workload deployments — and every result ends up in one place



1 Client → Service
2 Service → deployment helper: create the harnessed workload
3 helper → harnessed workload
4 Service → harnessed workload: run and monitor benchmark execution
5 Service → Experiment Tracker: emit the trace
6a workload → Telemetry Collector · 6b collector → Tracker
7 Experiment Tracker → Service: read the records back
8 Service → shared store: export the run's artifacts

3. Architecture

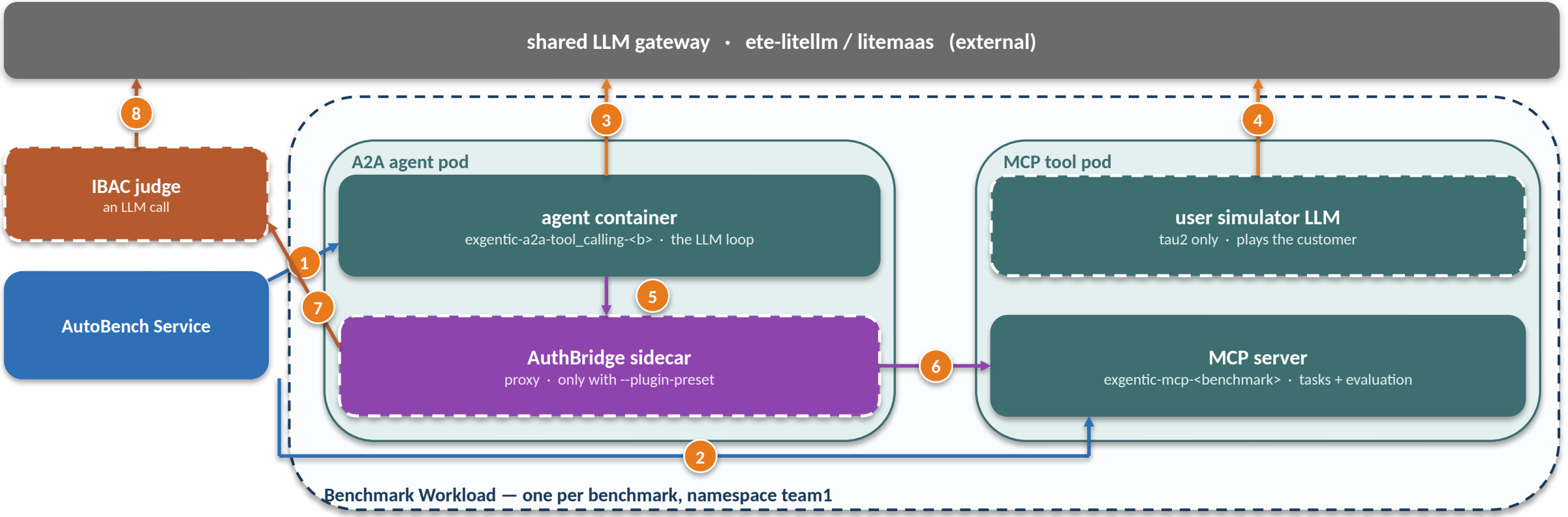
Relations between client, service, cluster-specific Keycloak / Rossoctl / workload, MLflow & S3



1 Client logs in to Keycloak (ROPC) → caller JWT
2 Client → Service: Authorization: Bearer <caller JWT>
3 Service validates JWT (JWKS) + mints its own benchmarker token (ROPC, via backchannel)
4 Service → Rossoctl: deploy / list agents + tools
5 Rossoctl creates the MCP tool + A2A agent in-cluster
6 Service drives the MCP session + the A2A call itself; the agent only issues execute_tool inside that session
7 Service emits the Agent.Session trace → MLflow (first)
8a A2A agent → OTEL collector · 8b collector → MLflow (optional workload spans — off by default)
9 Service reads back Agent.Session traces (MLflow)
10 Service exports run.json / report.* (S3)

4. Architecture with Workload Specific Components

Inside the Benchmark Workload — what every benchmark has, and what only some of them add



Which components a benchmark actually gets

benchmark	MCP tool image	LLMs per task	tool calls
gsm8k	exgentic-mcp-gsm8k	1 (agent)	~1
tau2	exgentic-mcp-tau2	2 (+ user simulator)	~11
appworld	exgentic-mcp-appworld	1 (agent)	~13
+ any preset	unchanged	+1 IBAC judge call per action	—

1 Service → agent: send_prompt, once per task
2 Service → MCP server: list_tasks, create_session, evaluate_session, delete_session (the verdict)
3 agent → gateway: 1 probe + N real chat calls
4 tau2 only: the user simulator is a 2nd LLM
5 with a preset: the agent's MCP calls go via the sidecar
6 sidecar → MCP server, once authorized
7 sidecar → judge, per isAction call
8 the judge is itself an LLM call
Dashed outline = present only in some configurations.
No preset: 5 + 6 become one direct agent → MCP call.

5. The Two-Token Auth Model

Why the caller's token is never forwarded upstream

Caller JWT (inbound)

Presented by the client as Authorization: Bearer.

- Used ONLY to attribute (preferred_username) and route (iss → instance)
- Signature validated via the issuer's JWKS
- aud / exp deliberately lenient in the dev/ops context
- **NEVER forwarded to Rossoctl**

benchmarker is special:

- A caller JWT with preferred_username == benchmarker authorizes config ops (GET/PUT /config)

Service token (outbound)

Minted by the Service per request via ROPC (password grant) using the instance's benchmarker credential.

- Always fresh → no expiry handling
- Presented to Rossoctl for all cluster-facing ops
- **The Service acts under its OWN identity, not the caller's**

Backchannel split (iss ≠ dialed host):

- When the iss host is unreachable (in-cluster kind), JWKS + token come from a configured backchannel URL; iss is matched, never dialed

Implication: at the Rossoctl / cluster layer every action appears as the Service's identity — so per-user attribution & audit live in the Service, keyed on (iss, preferred_username).

6. Benchmark Catalog & Run Lifecycle

Catalog (static registry)

gsm8k
single-turn · needs hf-secret + openai-secret

tau2
multi-turn · user-simulator LLM runs server-side in the MCP pod · raise timeout

appworld
long-horizon across simulated apps · runs e2e · raise task_timeout_seconds

Lifecycle (REST)

POST /benchmarks/{n}/deploy
create MCP tool + A2A agent → 201

GET /benchmarks/{n}/status
poll until tool_ready & agent_ready

POST /benchmarks/{n}/runs
precheck → 202 run_id (409 not deployed · 424 secret missing)

GET .../runs/{id}
poll pending → running → succeeded / failed

GET .../runs/{id}/report
MLflow records + artifacts (409 if MLflow unset)

download from S3
run.json · report.* · token_report.* · span_report.* · manifest.json

7.1 The Three Benchmarks — a Difficulty Ladder

Not interchangeable suites: each costs ~an order of magnitude more than the last

	gsm8k	tau2	appworld
What it tests	multi-step arithmetic	multi-turn dialogue + tools	long-horizon app automation
Tasks measured	172	60 (50 clean)	40 (35 clean)
Pass rate	0.98	0.83	0.00
Input tokens / task	343	82,966	210,792
Output tokens / task	205	1,955	21,499
LLM calls / task	2.1	11.4	23.6
Tool calls / task	1.1	11.2	13.4
Median task latency	5.3 s	92 s	232 s
Slowest task seen	62 s	425 s	581 s
Model we use	gpt-5-mini	claude-sonnet-5	gemini-2.5-pro
Task pool	8.5K (HuggingFace)	114 (retail domain)	grouped scenarios

The scale gap is the headline: a tau2 task costs ~240x the input tokens of a gsm8k task, an appworld task ~615x. A 50-task gsm8k run is minutes; a 20-task appworld run is half an hour and millions of tokens. Budget by benchmark, not by task count.

7.2 What Each Benchmark Stresses

Why all three are in the matrix, and what a result from each does and does not tell you

gsm8k — the canary

- One prompt in, one answer out — no dialogue partner.
- ~1 real LLM call + 1 tool call: the simplest agentic loop.
- Stresses almost nothing about the platform — which is the point.
- A failure here means INFRASTRUCTURE: deploy, auth, LLM reach, telemetry.
- Saturated at ~1.0, so it cannot discriminate models. Never read it as one.
- Deterministic enough that task 0 costs 320 input tokens on every cluster — we use that to prove two environments are comparable.

tau2 — the discriminator

- Multi-turn: a server-side USER SIMULATOR LLM plays the customer.
- Two models talk to each other, plus ~11 tool calls per task.
- Domain is retail (114 tasks) — the library default, never recorded in artifacts.
- The only leg where model choice dominates: 0.1 with gpt-5-mini vs 0.9 with claude-sonnet-5 on the SAME 10 tasks.
- Episodes are nondeterministic: across 6 samples of the same 10 tasks, 4 flip.
- At n=10 one task moves pass_rate by 0.10 — it cannot resolve less than that.

appworld — the stress test

- Realistic chores across simulated apps: discover APIs, chain many calls.
- ~24 LLM calls, ~13 tool calls, ~211k input, ~4 min per task.
- Evaluation is PROGRAMMATIC unit tests over final app state — no LLM judge.
- Pass rate 0.0 is honest, not broken: runs complete and tokens record — the agent just does not finish the job.
- ~94% of tasks call finish: it believes it is done and the assertions disagree. No verification pass.
- Its job here is pipeline stress: long contexts, big traces, real timeouts.

If you want to ...	use
check a cluster / deploy / auth / telemetry path works	gsm8k, 1–10 tasks
exercise concurrency and volume cheaply	gsm8k, 50 tasks at max_parallel_sessions=4
compare models meaningfully	tau2 — it discriminates; gsm8k saturates at ~1.0
stress long contexts, long tasks, timeouts	appworld

7.3 Reading the Numbers — Five Things That Mislead

Every one of these cost us a wrong conclusion first

1. pass_rate = evaluated_pass / total

A task that ERRORS before evaluation counts as not passed, so a low rate can mean “failed the task” or “never got judged”. Check the error column too.

2. The llm column overcounts real calls by one

Every task issues an extra max_tokens=1 capability probe, counted as a chat span. llm=2 on gsm8k means ONE real call.

3. Implausibly small tokens = lost telemetry, and tokens == 0 will not catch it

When a usage-bearing span is lost, what survives is the probe — whose own usage is 0/0 on reasoning models but 8/1 on claude-sonnet-5 and 1/0 on gemini. Use the structural test: llm <= 1 with tool >= 2 is impossible.

4. Output varies MORE than input, in most runs

Measured OUT CV > IN CV in 27 of 33 legs. gsm8k's prompt is near-constant while answer length swings; only long-horizon appworld inverts it. Do not infer a direction — read the CV.

5. Task selection is deterministic

A run takes the first max_tasks tasks, so the same task_id is the same task across runs and clusters, and a smaller run is a prefix of a larger one. That is what makes cross-platform comparison like-for-like — six legs matched to the byte.

8.1 The Canonical 12-Run Matrix

One fixed set of 12 request bodies — the same on every platform, every version

#	benchmark	tasks	parallel	what it varies
1	gsm8k	1	1	baseline — the smoke test
2	gsm8k	10	1	volume, still serial
3	gsm8k	50	4	volume + concurrency
4	gsm8k	5	4	model swap → Azure/gpt-4.1
5	gsm8k	5	4	AuthBridge preset: auth-only
6	gsm8k	5	4	AuthBridge preset: ibac-only
7	gsm8k	5	4	AuthBridge preset: full (enforce)
8	gsm8k	5	4	full + per-plugin override ibac:observe
9	tau2	10	1	multi-turn + user simulator
10	tau2	20	4	multi-turn under concurrency
11	appworld	5	1	long-horizon, gemini-2.5-pro
12	appworld	20	4	long-horizon under concurrency

Why a FIXED matrix: task selection is deterministic, so the same leg run anywhere executes the same tasks in the same order. That is what makes a difference attributable to the platform rather than to the workload — and it is why every leg deploys fresh, since reusing a warm agent silently loses telemetry. Driven by one command (reference/run-12.py, ~1h55m); the three generated documents in results/ derive every number from the mirrored artifacts.

8.2 What the 12 Runs Measured

v1.24 on OpenShift (Service on ykt3, workloads on ykt2) — 138 tasks, all 12 legs succeeded

#	bench	pass	wall	input tokens
1	gsm8k	1.00	9 s	320
2	gsm8k	1.00	42 s	3,166
3	gsm8k	1.00	97 s	15,684
4	gsm8k	0.80	26 s	3,613
5	gsm8k	1.00	31 s	1,564
6	gsm8k	1.00	35 s	1,564
7	gsm8k	1.00	32 s	1,564
8	gsm8k	1.00	30 s	1,564
9	tau2	0.70	780 s	852,240
10	tau2	0.85	425 s	1,550,718
11	appworld	0.00	2164 s	1,988,986
12	appworld	0.00	1769 s	4,009,033

All eight gsm8k legs pass
1.00 everywhere except #4, where ONE task of five fails on the gpt-4.1 swap. The four AuthBridge presets do not change the result — only the latency.

tau2 is the only leg that moves
0.70 and 0.85. Across six samples of the same 10 tasks, four flip: at n=10 one task is worth 0.10, so this cannot resolve less than that.

appworld 0.00 is the honest result
Runs complete, tokens record, evaluation says the goal was not met. ~94% of tasks self-declare finish.

Token attribution complete: 0 of 138 rows lost
The previous matrix had 15 damaged rows that a tokens == 0 check reported as clean. Fresh deploy per leg, plus a structural detector.

8.4M input / 674k output tokens
over 5,438 s of wall time — appworld alone is 71% of the input, on 25 of 138 tasks.

9.1 OpenShift vs KinD — Like-for-Like

Same 12 request bodies, same Service version, verified-identical instance config

#	bench	pass OCP	pass KinD	in OCP	in KinD	
1	gsm8k	1.00	1.00	320	320	identical
2	gsm8k	1.00	1.00	3,166	3,166	identical
3	gsm8k	1.00	1.00	15,684	15,684	identical
4	gsm8k	0.80	0.80	3,613	4,184	
5	gsm8k	1.00	1.00	1,564	1,564	identical
6	gsm8k	1.00	1.00	1,564	1,564	identical
7	gsm8k	1.00	1.00	1,564	1,564	identical
8	gsm8k	1.00	1.00	1,564	1,564	identical
9	tau2	0.70	0.70	852,240	847,530	
10	tau2	0.85	0.80	1,550,718	1,552,641	
11	appworld	0.00	0.00	1,988,986	1,150,541	
12	appworld	0.00	0.00	4,009,033	3,646,292	

11 of 12 pass rates identical
Only #10 differs, by one task of twenty (0.85 vs 0.80) — and tau2 is the leg we know flips run to run on either platform.

7 legs byte-identical on input tokens
320 / 3,166 / 15,684 / 1,564 x4. Deterministic task selection plus the same model means identical work — the strongest available proof this is a like-for-like comparison, not merely a similar one.

Wall time is a dead heat
5,438 s on OpenShift vs 5,429 s on a single KinD node — 0.2% apart in total, though individual legs differ by up to 8x.

0 lost-attribution rows on both sides
138 OCP tasks, 135 KinD tasks.

So the platform is not the variable
Where the two differ it traces to episode nondeterminism (tau2), small samples (#4 = one task), or appworld task timeouts — not to OpenShift vs KinD.

9.2 What AuthBridge Costs

Legs #3 and #5–8 ran byte-identical work, so latency differences are the plugin config

component	cost	measured from
Sidecar present at all	~+9.5 s / task (OCP)	flat across all four presets
Tool call, sidecar but unjudged	+0.50 s	auth-only
Tool call, judged	+1.96 s on top	full + observe (all 5 judged)

	OCP	KinD	ratio
connect_mcp with sidecar	3131 ms	168 ms	18.6x
tool call, unjudged	545 ms	13 ms	41.1x
tool call, judged	2507 ms	1167 ms	2.1x

The judge is itself an LLM call
The sidecar POSTs to a chat-completions endpoint per authorized action, which is why it costs seconds and why its latency is so variable.

Most of the OCP figure is topology, not plugins
The judged-call cost is comparable across platforms (2.1x) because it is dominated by a shared external gateway. The fixed cost is 8x apart because this OCP matrix is cross-cluster — token exchange over external routes. Do NOT quote +9.5 s as “the cost of AuthBridge”; the single-node figures (~+1.2 s fixed, ~+1.1 s per judged call) are the better estimate.

Presets cannot be ranked from these numbers
Only 1 of 6 IBAC legs authorized all of its tool calls: OCP judged 4/5, 3/5, 5/5 and KinD 3/5, 4/5, 3/5. A leg that judged fewer calls shows a lower median — a mixture, not a saving. Whether that is decision caching under concurrency or a fail-open gap is still OPEN; the test is one ibac-only leg at parallelism 1.

Latency only
The sidecar's CPU and memory cost is unmeasured — every infra field in these runs is 0.0.

10. What the Service Can & Cannot Enact

The HTTP-only boundary, made explicit

✓ Enacts over HTTP

- Deploy MCP tool + A2A agent (CPU/mem, image, env)
- Deploy-time model swap (per-experiment agent)
- authbridge_enabled → inject the sidecar (layer-2)
- plugin_preset / plugins / on_error → AuthBridge layer-3
  → pluginPreset/plugins/onError; operator renders ConfigMap
- Run benchmark sessions; collect pass/fail + latency
- Emit + read MLflow traces; export to S3
- Service-owned config: MLflow + S3 via PUT /config
  benchmarker only

✗ Out-of-band (reports, doesn't do)

- Cluster Secrets (hf-secret, openai-secret)
  operator provisions; run precheck returns 424 naming it
- AuthBridge CLUSTER config — ibac judgeEndpoint / judgeModel
  in the platform-config ConfigMap, not the agent API
- Any cluster-level API call — Rossoctl does these server-side

Principle:

- Never silently ignore an un-enactable request —
  precheck (424) or reject (422), with an actionable reason

Cluster-agnostic by construction: works on kind / vanilla k8s / OpenShift; cross-cluster runs use per-instance route templates + a reachable internal issuer.